231501506

Ex 4: Comparative Analysis of Large Language Models

Learning Objective:
Develop a Python program that performs a comparative analysis of Large Language Models by analyzing the text-generation performance of OpenAI, Gemini, and Hugging Face models.

Steps:

1. Import Required Libraries (OpenAI, HuggingFace Transformers, Gemini API)
2. Configure OpenAI key and Google Gemini API key. (This key should be personal. Do not share with anyone).
3. Initialize the Models (OpenAI, Gemini, and Hugging Face).
4. Write function to send prompt and receive response from OpenAI, Gemini, and Hugging Face models
5. Provide a common text prompt. (e.g., “Write a haiku about AI and nature”).
6. Generate Outputs by sending the same prompt to all three models and collect their responses.
7. Display the Model Responses individually. Use Markdown / HTML formatting to display results clearly.
8. Construct an evaluator. (For evaluator you can use Gemini) and produce ranks.
9. Display the ranked model names in best → worst order.

CODE:

```
import google.generativeai as genai
from transformers import pipeline
from huggingface_hub import login
import time
genai.configure(api_key="AIzaSyBQKa35UnFwVyCNGuo6knGlqberpTPxKzM")
login("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")
gemini_model = genai.GenerativeModel("gemini-2.5-flash")
tiny_model = pipeline(
    "text-generation",
    model="TinyLlama/TinyLlama-1.1B-Chat-v1.0"
)
hf_model = pipeline(
    "text-generation",
    model="distilgpt2"
)
def query_gemini(prompt):
```

```
    start = time.time()
    res = gemini_model.generate_content(prompt)
    return res.text, time.time() - start

def query_tinyllama(prompt):
    start = time.time()
    res = tiny_model(prompt, max_length=80,
do_sample=True,temperature=0.7)
    return res[0]['generated_text'], time.time() - start

def query_hf(prompt):
    start = time.time()
    res = hf_model(prompt, max_length=80, do_sample=True,temperature=0.7)
    return res[0]['generated_text'], time.time() - start
prompt = "Write a haiku about AI and nature"
gemini_out, gemini_time = query_gemini(prompt)
tiny_out, tiny_time = query_tinyllama(prompt)
hf_out, hf_time = query_hf(prompt)
from IPython.display import Markdown, display
display(Markdown(f"""
## Gemini
Time: {gemini_time:.2f}s
{gemini_out}
---
## TinyLlama (HF)
Time: {tiny_time:.2f}s
{tiny_out}
---
## Hugging Face (DistilGPT2)
Time: {hf_time:.2f}s
{hf_out}
"""))
eval_prompt = f"""
Evaluate the outputs based on:
- Relevance
- Creativity
- Clarity
Prompt: {prompt}
Gemini:
{gemini_out}
TinyLlama:
{tiny_out}
HuggingFace:
```

231501506

```
{hf_out}
Return ranking BEST to WORST.
"""
ranking = gemini_model.generate_content(eval_prompt).text
display(
    n(f"""
# Final Ranking

{ranking}
"""))
```

OUTPUT:

```
Loading weights: 100% ■■■
```

231501506